

Aprendizagem de Máquina – IF699

Alice Peruniz de Oliveira <apo2@cin.ufpe.br>
Ariel Rodrigues Sousa dos Santos <arss5@cin.ufpe.br>
Felipe Torres de Macedo <ftm2@cin.ufpe.br>
Victor Medonça Aguiar <vma3@cin.ufpe.br>

2026.2

1 Introdução

Este relatório descreve a atividade desenvolvida para a disciplina de Aprendizagem de Máquina, cujo objetivo era explorar na base de dados *Spambase* uma técnica de agrupamento e um conjunto de classificadores supervisionados. O rótulo original de spam/não-spam da base não foi usado, e no lugar dele as classes foram definidas a partir de um agrupamento com K-Means, de modo que os clusters encontrados passam a fazer o papel da variável que os classificadores tentam prever.

Na prática, isso significou rodar o K-Means para diferentes números de clusters e escolher o melhor deles através do coeficiente de silhueta, treinar cinco classificadores (Árvore de Decisão, Naive Bayes, Regressão Logística, KNN e Random Forest) sobre esses clusters buscando os melhores hiperparâmetros de cada um, e por fim verificar como cada classificador se comporta conforme a quantidade de dados de treino varia, através de curvas de aprendizagem. As seções seguintes descrevem a base de dados, a metodologia usada em cada etapa e os resultados obtidos.

2 Base de Dados

A base *Spambase*, do UCI Machine Learning Repository, reúne 4601 e-mails descritos por 57 atributos contínuos, sem valores faltantes. A maior parte deles (48 atributos) mede a frequência relativa de determinadas palavras no corpo do e-mail (*word_freq_**); outros 6 medem a frequência de caracteres específicos, como \$ e ! (*char_freq_**); e os 3 últimos capturam o comprimento de sequências de letras maiúsculas (média, máximo e total), um índice típico de e-mails de spam. A base também traz um rótulo binário de spam (1813 exemplos) e não-spam (2788 exemplos), que aqui serve apenas de referência para contextualizar a base, já que, como explicado na introdução, esse rótulo não é usado como alvo de predição em nenhuma etapa deste trabalho.

Como os atributos têm escalas bastante distintas entre si, todos foram padronizados (média 0 e desvio-padrão 1) antes de qualquer modelagem. Essa normalização é importante tanto para o K-Means, cuja noção de distância seria dominada pelas variáveis de maior escala, quanto para classificadores como KNN e Regressão Logística, sensíveis à magnitude dos atributos.

3 Metodologia

3.1 Definição das classes por agrupamento

Para escolher um número de clusters adequado, o K-Means foi executado 100 vezes para cada $k \in \{2, 3, 4, 5\}$, cada execução partindo de uma inicialização aleatória diferente. Dentro de cada valor de k , mantivemos apenas o modelo de menor inércia (soma das distâncias quadráticas ao centróide mais próximo), reduzindo a chance de reportar um resultado ruim causado por uma inicialização infeliz. Para o melhor modelo de cada k , calculamos o coeficiente de silhueta e escolhemos

$$k^* = \arg \max_k \text{Sil}(k),$$

isto é, o número de grupos que produz a separação mais nítida entre clusters. Os rótulos de cluster do modelo vencedor tornam-se a variável resposta usada em todo o restante do trabalho.

3.2 Protocolo de treinamento e comparação dos classificadores

Uma comparação entre classificadores só é informativa se todos forem avaliados sob as mesmas condições, então os cinco modelos seguiram exatamente o mesmo protocolo:

- os dados foram divididos de forma estratificada em 80% para treino e 20% para teste, preservando em ambos os conjuntos a proporção entre clusters;
- o conjunto de teste foi isolado a partir desse ponto e reservado exclusivamente para a avaliação final, sem influenciar em nenhum momento o treinamento ou a escolha de hiperparâmetros;
- para cada classificador, um conjunto de hiperparâmetros candidatos foi avaliado por validação cruzada estratificada de 5 folds dentro do conjunto de treino (GridSearchCV), usando o F1 ponderado como critério de seleção, métrica escolhida por ser mais informativa que a acurácia diante do desbalanceamento entre clusters, discutido na Seção 4.1;
- o melhor modelo encontrado por essa busca foi então avaliado uma única vez sobre o conjunto de teste, reportando-se precision, recall e F1-score.

Como a partição treino/teste e a estratégia de validação cruzada são idênticas para todos os cinco classificadores, diferenças observadas nos resultados finais podem ser atribuídas ao modelo em si, e não a alguma vantagem de protocolo.

3.3 Curvas de aprendizagem

Para avaliar a robustez dos resultados frente à quantidade de dados de treino, cada classificador foi retreinado do zero, já com os hiperparâmetros fixados pela busca anterior, para 19 proporções treino/teste diferentes, variando de 5%/95% a 95%/5% em passos de 5%, sempre com amostragem estratificada. Em cada proporção, calculamos precision, recall e F1-score tanto no conjunto de treino quanto no de teste daquela divisão específica. Comparar as duas curvas permite identificar sinais de *overfitting* (quando o desempenho de treino é bem superior ao de teste) e observar a partir de que fração de dados cada modelo estabiliza seu desempenho.

4 Resultados e Discussão

4.1 Agrupamento e escolha do número de clusters

A Tabela 1 e a Figura 1 resumem os resultados do K-Means para cada valor de k testado.

Table 1: Melhor inércia (entre 100 execuções) e silhueta por valor de k .

k	Inércia	Silhueta
2	242529.07	0.6596
3	231547.96	0.1251
4	223997.44	0.1582
5	218500.30	0.1610

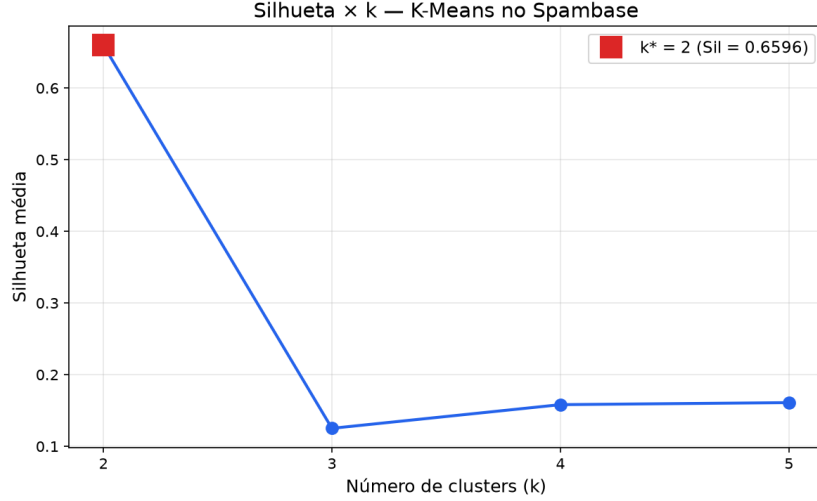


Figure 1: Silhueta em função de k . O ponto em destaque indica $k^* = 2$.

A inércia decresce de forma monotônica com k , o que é esperado, já que mais clusters sempre reduzem a distância intra-cluster, e por isso não serve como critério de escolha. A silhueta, por outro lado, é claramente maximizada em $k^* = 2$ (0.6596), bem acima dos demais valores, o que sugere que a base é mais naturalmente descrita por dois grandes grupos do que por partições mais finas. Um efeito colateral dessa escolha é que os clusters resultantes ficam fortemente desbalanceados: 4567 e-mails (99.3%) caem no cluster 1 e apenas 34 (0.7%) no cluster 2. Esse desbalanceamento é o motivo pelo qual optamos pelo F1 ponderado, e não pela acurácia, como critério para a busca de hiperparâmetros e para a comparação entre classificadores.

4.2 Comparação entre os classificadores

Com $k^* = 2$, a tarefa passa a ser uma classificação binária sobre 3680 exemplos de treino e 921 de teste, mantendo em ambos a mesma proporção de 99.3%/0.7% entre clusters. A Tabela 2 mostra o espaço de hiperparâmetros explorado para cada classificador; a Tabela 3, a combinação vencedora de cada busca, junto com o F1 médio obtido na validação cruzada (calculado inteiramente sobre o treino, sem qualquer contato com o teste).

Table 2: Espaço de hiperparâmetros avaliado por classificador (validação cruzada estratificada de 5 folds, otimizando F1 ponderado).

Classificador	Hiperparâmetros testados	Combinações
Árvore de Decisão	<code>max_depth</code> : {3, 5, 10, 20, None}; <code>min_samples_split</code> : {2, 5, 10, 20}; <code>min_samples_leaf</code> : {1, 2, 5, 10}; <code>criterion</code> : {gini, entropy}	160
Naive Bayes	<code>var_smoothing</code> : 7 valores em escala logarítmica entre 10^{-12} e 10^{-6}	7
Regressão Logística	<code>C</code> : {0.01, 0.1, 1, 10, 100}; <code>penalty</code> : {l1, l2}; <code>solver</code> : {liblinear, saga}; <code>class_weight</code> : {None, balanced}	40
KNN	<code>n_neighbors</code> : {1, 3, 5, 7, 9, 11, 15, 21}; <code>metric</code> : {euclidean, manhattan, chebyshev}; <code>weights</code> : {uniform, distance}	48
Random Forest	<code>n_estimators</code> : {50, 100, 200}; <code>max_depth</code> : {5, 10, 20, None}; <code>min_samples_split</code> : {2, 5, 10}; <code>min_samples_leaf</code> : {1, 2, 5}; <code>class_weight</code> : {None, balanced}	216

Table 3: Melhor combinação de hiperparâmetros por classificador e F1 médio na validação cruzada.

Classificador	Melhores hiperparâmetros	F1 (CV)
Árvore de Decisão	<code>criterion=entropy, max_depth=3, min_samples_leaf=1, min_samples_split=2</code>	0.9992
Naive Bayes	<code>var_smoothing=1e-07</code>	0.9700
Regressão Logística	<code>C=0.1, class_weight=None, penalty=l2, solver=liblinear</code>	0.9997
KNN	<code>metric=euclidean, n_neighbors=15, weights=distance</code>	1.0000
Random Forest	<code>class_weight=balanced, max_depth=5, min_samples_leaf=1, min_samples_split=2, n_estimators=50</code>	0.9997

Um detalhe interessante é que a busca encontrou modelos bem simples como melhores: a árvore de decisão ideal tem profundidade máxima 3, e a floresta aleatória prefere balancear as classes (`class_weight=balanced`) a ficar mais complexa. Isso faz sentido, pois se os clusters já estão bem separados, não é preciso um modelo elaborado para distingui-los.

A Tabela 4 e a Figura 2 trazem o desempenho de cada classificador no conjunto de teste, isolado durante toda a busca de hiperparâmetros.

Table 4: Métricas ponderadas no conjunto de teste, por classificador.

Classificador	Precision	Recall	F1-score
Árvore de Decisão	1.0000	1.0000	1.0000
Random Forest	1.0000	1.0000	1.0000
KNN	1.0000	1.0000	1.0000
Regressão Logística	0.9991	0.9989	0.9990
Naive Bayes	0.9919	0.9327	0.9588

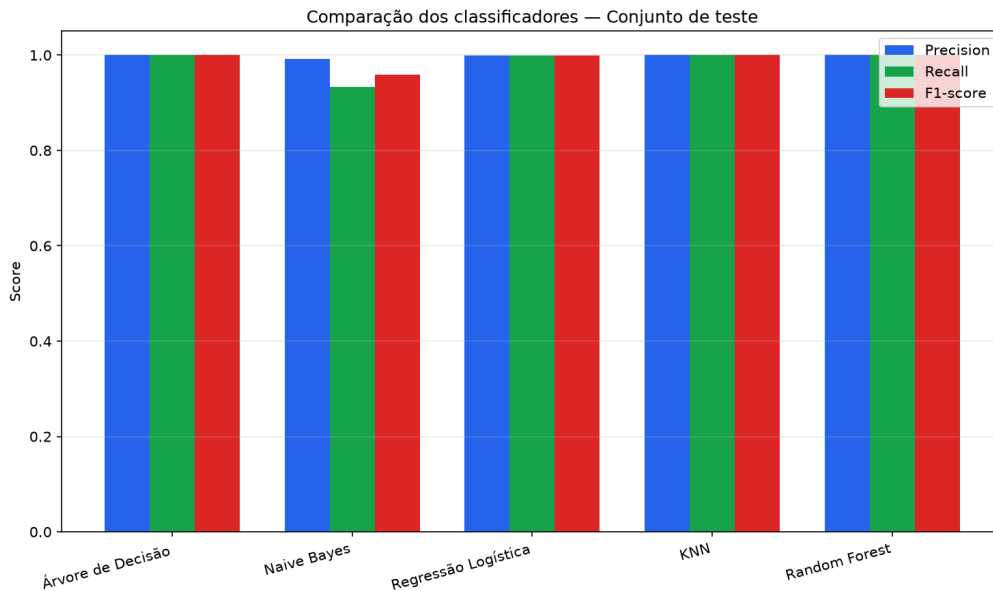


Figure 2: Comparação de precision, recall e F1-score no conjunto de teste para os cinco classificadores.

Árvore de Decisão, Random Forest e KNN atingem desempenho perfeito no teste, e a Regressão Logística fica muito próxima disso (F1 = 0.9990). O Naive Bayes se destaca como o único claramente inferior, com recall de apenas 0.9327, porque sua premissa de independência entre atributos é pouco realista para 57 features de frequência de palavras, que tendem a ser correlacionadas entre si (e-mails com muitas palavras de spam costumam repetir várias delas simultaneamente).

4.3 Curvas de aprendizagem

As Figuras 3, 4 e 5 mostram como precision, recall e F1-score evoluem no treino (linha tracejada) e no teste (linha sólida) conforme a fração de dados usada para treinar cada classificador aumenta de 5% para 95%.

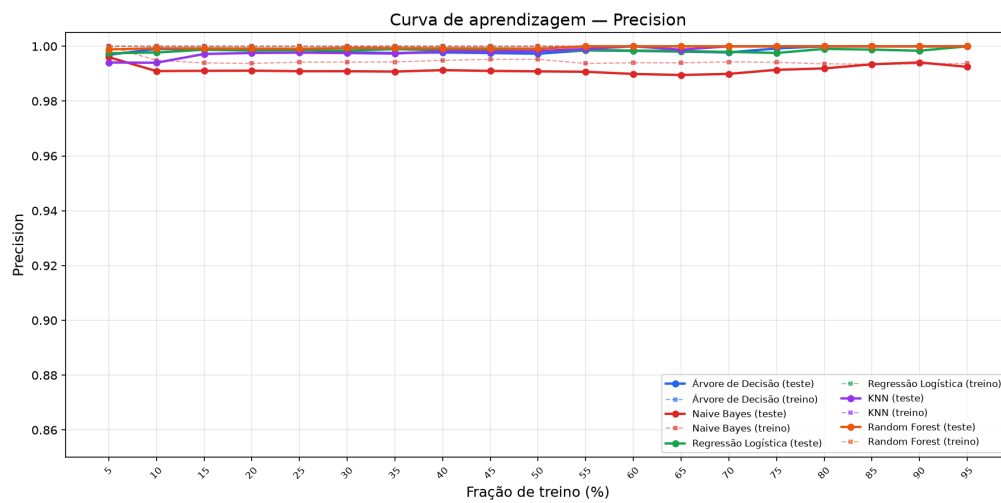


Figure 3: Curva de aprendizagem para a métrica Precision.

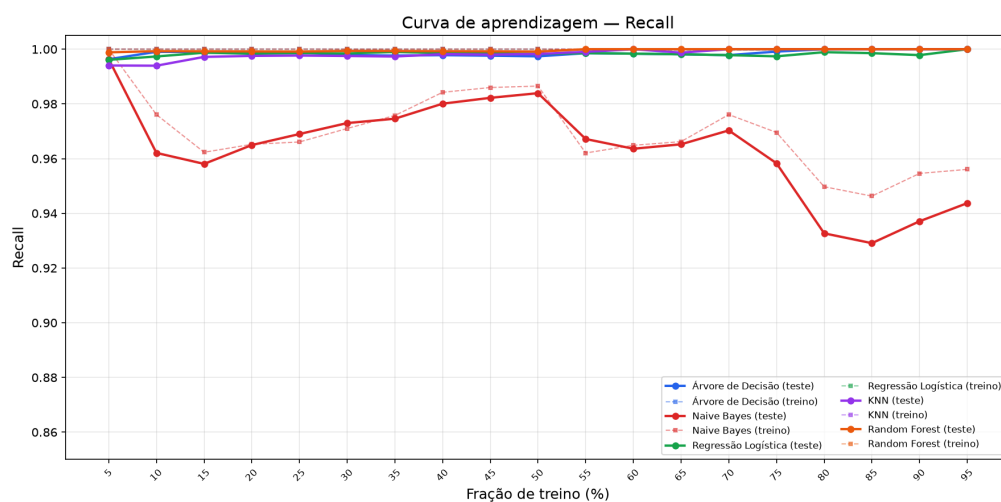


Figure 4: Curva de aprendizagem para a métrica Recall.

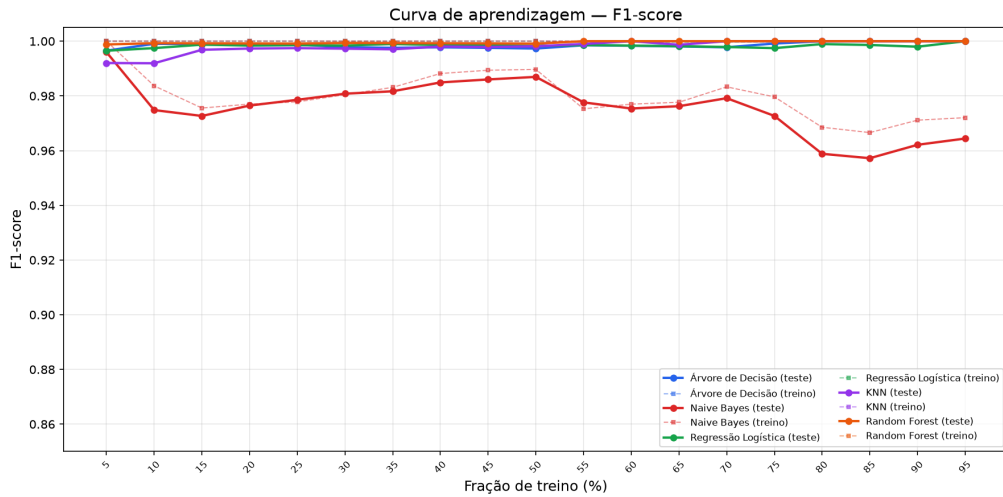


Figure 5: Curva de aprendizagem para a métrica F1-score.

O padrão mais evidente é que a maioria dos classificadores atinge desempenho quase perfeito com uma fração pequena de treino, algo em torno de 10% a 15% dos dados, porque os clusters usados como classes foram gerados pelo próprio K-Means, algoritmo que por construção maximiza a separação entre grupos, resultando em uma fronteira de decisão simples no espaço normalizado. É por isso também que a silhueta ficou relativamente alta, em 0.66.

Árvore de Decisão, KNN e Random Forest convergem rapidamente para F1 próximo de 1.0, com as curvas de treino e teste praticamente sobrepostas, sem indício de *overfitting*. A Regressão Logística tem comportamento semelhante, mas fica ligeiramente abaixo do desempenho perfeito (0.999 em vez de 1.0), já que, como modelo linear, ela tem mais dificuldade em ajustar exatamente a fronteira quando alguns exemplos do cluster minoritário (apenas 34 no total) ficam muito próximos dela.

O Naive Bayes é o único caso em que treino e teste não convergem completamente, e seu recall no teste permanece consistentemente mais baixo (em torno de 0.93) mesmo com frações grandes de treino. Isso indica um viés estrutural do modelo, e não falta de dados, já que a suposição de independência entre atributos limita seu desempenho independentemente de quanto treino ele recebe.

Por fim, um ponto interessante é como os modelos lidam com o forte desbalanceamento entre clusters. Com apenas 34 exemplos do cluster minoritário na base inteira, frações de treino de 5% deixam disponíveis apenas 1 ou 2 exemplos dessa classe, e ainda assim KNN e Random Forest conseguem classificá-los corretamente, o que se explica pelo uso de `weights=distance` e `class_weight=balanced`, respectivamente, ambos pensados justamente para compensar esse tipo de desbalanceamento.

5 Conclusão

No fim, o K-Means com $k^* = 2$ separou a base em dois grupos bem distintos, e essa separação acabou sendo fácil demais para a maioria dos classificadores. Árvore de Decisão, KNN e Random Forest acertaram tudo no teste, a Regressão Logística ficou bem perto disso, e só o Naive Bayes teve um desempenho visivelmente pior, por conta da sua suposição de independência entre atributos, que não se aplica muito bem aqui.

No entanto, vale ressaltar que esse desempenho quase perfeito tem uma explicação simples. Como as próprias classes foram criadas pelo K-Means, um algoritmo que por definição tenta separar os grupos o máximo possível, era esperado que a classificação depois fosse relativamente tranquila. Isso não invalida a comparação entre os classificadores, já que todos enfrentaram o mesmo problema e sob as mesmas condições, mas mostra que esses números não devem ser interpretados como “os classificadores resolveriam qualquer problema de spam com essa facilidade”. Com o rótulo original de spam da base, por exemplo, provavelmente os resultados seriam mais próximos do mundo real, com mais diferença entre os modelos.